

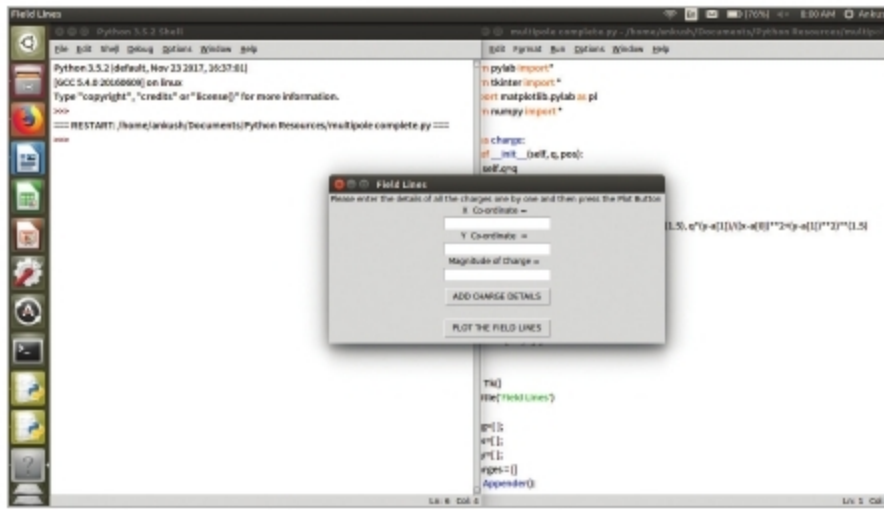


Figure 1: Output of the program after its first execution

A class is a data type that has its methodology of functioning and its attributes are defined by the programmer. Whenever a class is created, it creates its own namespace, which is a set of names pertaining to that class. Whenever that class is then called by the programmer, it is termed an instantiation of that class.

A method is any attribute (function or object) that already belongs to the class and is accessed using an attribute reference. Usually, when creating an instance of a class in a program, the class has some initial values for the internal data and this is done by defining the `__init__` method.

For example:

```
class Charge:
    def __init__(self, x, y, mag):
        self.x = x
        self.y = y
        self.mag = mag
```

For a field line plot, we require the descriptions of charges like position and magnitude (putting the + or – sign will take care of the type of charge), which were taken from the user with the help of a GUI built using the Tkinter module in Python.

```
from pylab import *
represents all submodules
from tkinter import *
import matplotlib.pyplot as pl
from numpy import *

class charge:
    def __init__(self, q, pos):
        self.q=q
        self.pos=pos

def E_point_charge(q, a, x, y):
    return q*(x-a[0])/((x-a[0])**2+(y-a[1])**2)**(1.5),
    q*(y-a[1])/((x-a[0])**2+(y-a[1])**2)**(1.5)

# formula to calculate electric field due to a single charge
```

```
def E_total(x, y, charges):
    Ex, Ey=0, 0
    for C in charges:
        E=E_point_charge(C.q, C.pos, x, y)
        Ex=Ex+E[0]
        Ey=Ey+E[1]
    return [ Ex, Ey]
```

```
w = Tk()
```

```
# creating a graphical user interface window. Figure 1.
w.title('Field Lines')
```

```
#The name for a graphical user interface window Figure 1.
```

```
mag=[ ];
```

```
#List to store the magnitude of charges.
```

```
plax=[ ];
```

```
#List to store the x coordinate of charges.
```

```
play=[ ];
```

```
#List to store the y coordinate of charges.
```

```
charges = []
```

```
def Appender():
```

```
    x1 = (eval(xv.get()))
    x1 = float(x1)
    plax.append(float(x1))
```

```
    y1 = (eval(yv.get()))
    y1 = float(y1)
    play.append(float(y1))
```

```
    mag1 = (eval(magv.get()))
    mag1 = float(mag1)
    mag.append(float(mag1))
    c = len(plax)
    pos=[];
    cx=float(plax[c-1]);
    pos.append(cx);
    cy=float(play[c-1]);
    pos.append(cy);
    cm=float(mag[c-1]);
    charges.append(charge (cm,pos))
```

```
space = Label(w, text = "Please enter the details of all
the charges one by one and then press the Plot Button
").pack()
Xentry = Label(w, text = "X Co-ordinate = " ).pack()
xv = StringVar()
xentry = Entry(w, text variable = xv , font=("Sitka
```